

# Cloud Segmentation

Fares Atef sec: 1 BN: 34, Ghaith Mohammed sec:1 BN: 33

Amr Mohammed sec: 1 BN: 32, Amr Magdy sec:1 BN: 31

Cairo University, Computer Engineering Department

## I. INTRODUCTION

This project focuses on exploring and comparing different techniques for segmenting clouds in satellite imagery. Rather than just training a model and submitting its outputs, the work also involved:

- Literature Review
- Exploratory Data Analysis
- Preprocessing
- Model Training
- Model Evaluation
- Performance Analysis Module

Through this document we will explore each phase in detail.

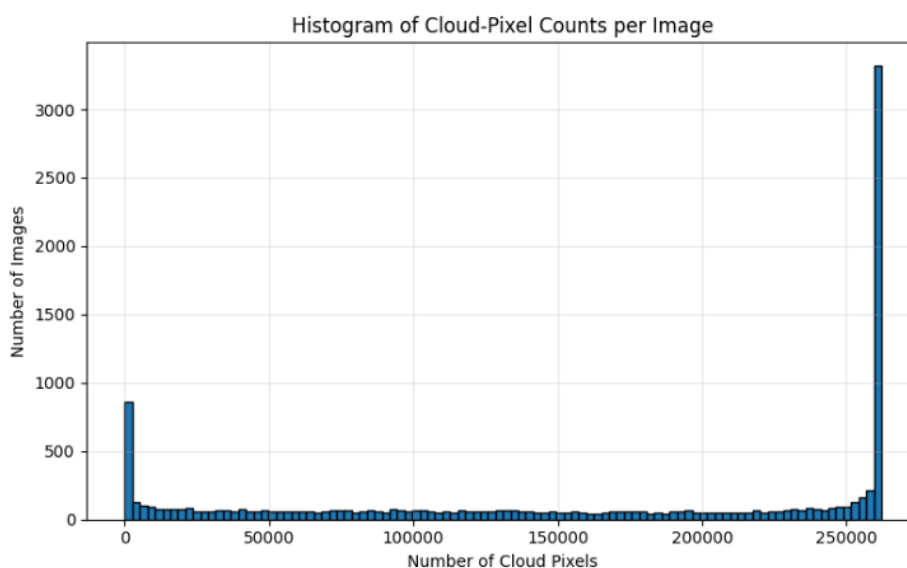
## II. LITERATURE REVIEW

To get started, we first researched the problem by searching online for previous work. We reviewed several survey papers that summarize and compare different cloud-segmentation approaches, then dug deeper to find additional studies those surveys didn't cover. Much of our methodology builds on a key survey, which you can find here: [link](#).

Multiple surveys consistently show that DeepLabV3+ outperforming other architectures by a clear margin. To push beyond that, we also investigated recent state-of-the-art semantic-segmentation techniques, many of which leverage transformer backbones, and tried a transformer-based model for cloud segmentation in our project. for segmentation

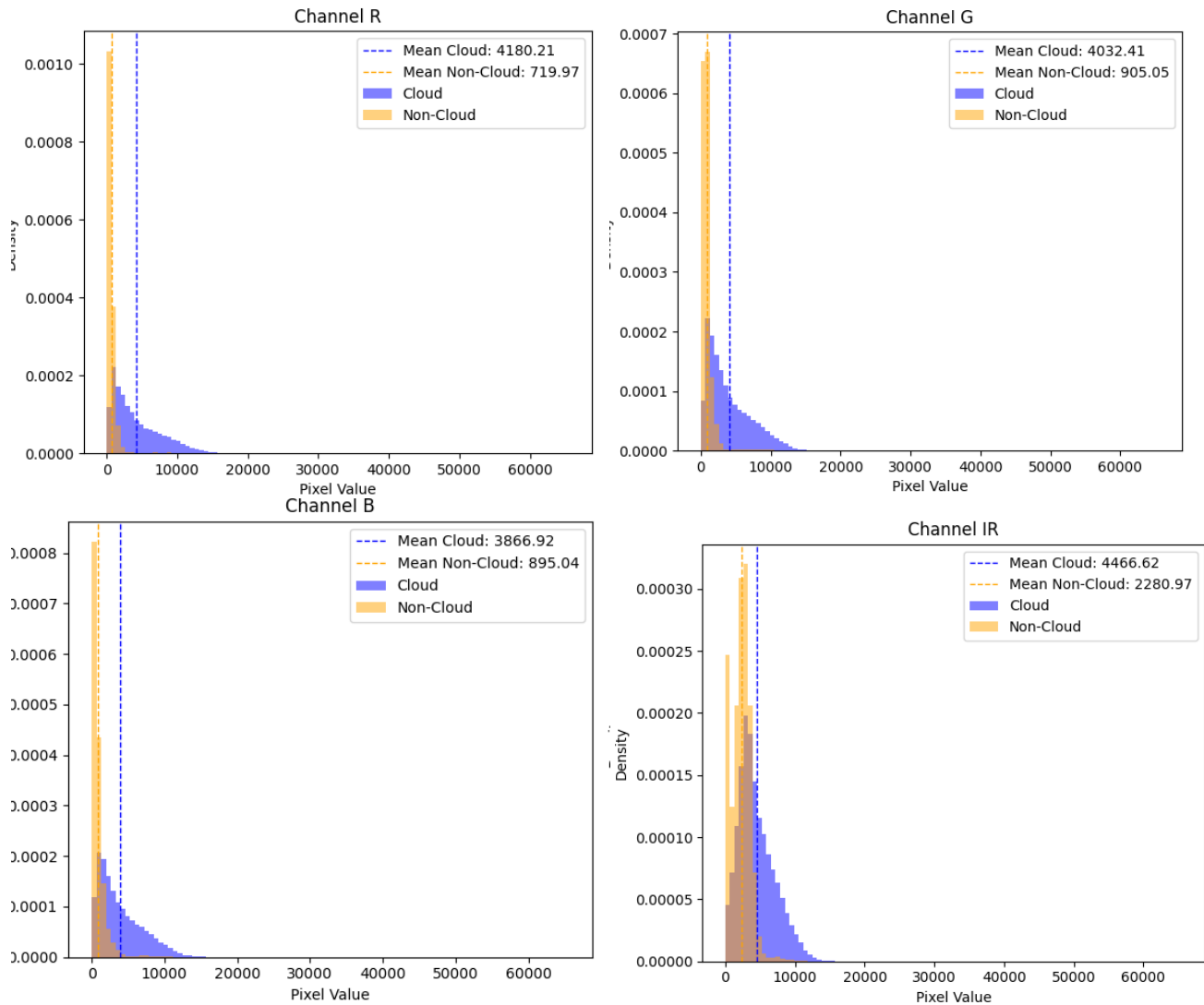
## III. EXPLORATORY DATA ANALYSIS

As an initial step, we examined the dataset's size, image dimensions, and pixel-mask distributions, and we visualized a number of individual samples. To illustrate how the mask values are distributed across the dataset, we generated the following histogram:

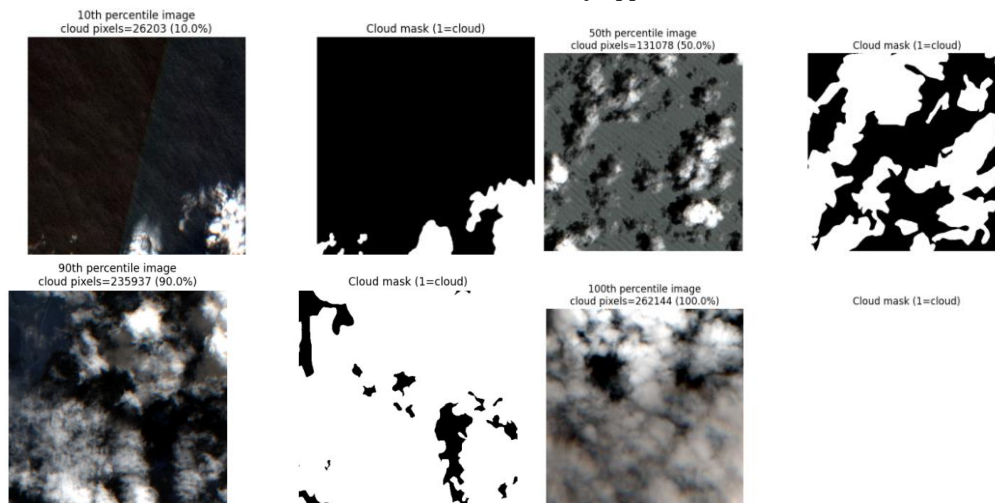


The histogram reveals a pronounced class imbalance: most images lie at the extremes—either almost entirely cloud-free or heavily overcast—with relatively few intermediate cases. Such a skewed distribution can hinder model training, as the network may become biased toward the majority classes and struggle to generalize to more balanced or moderate cloud cover.

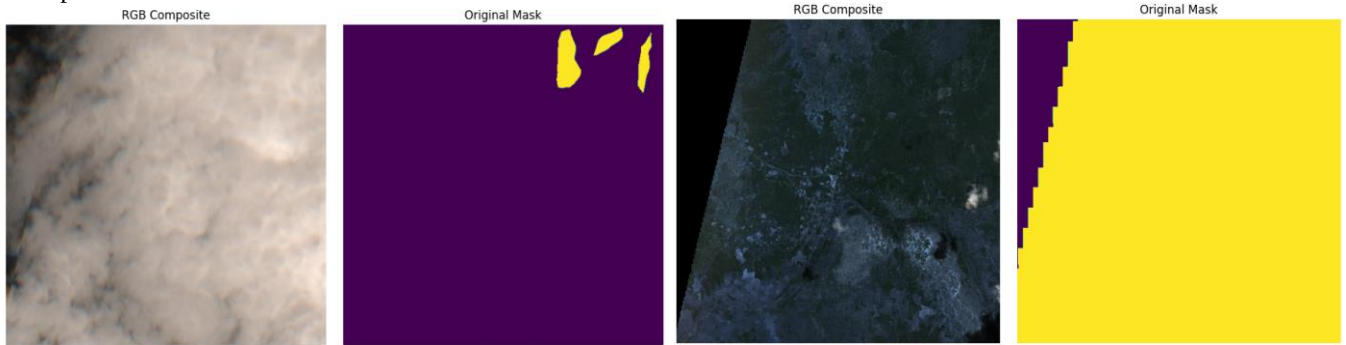
To dive deeper, we plotted the pixel-value distributions for each spectral band, separating cloud (mask) and non-cloud (background) pixels. The resulting per-channel histograms, shown below, highlight how each band's intensity differs between masked and unmasked regions.



Analysis of the per-band pixel-value distributions shows that the cloud pixels (the mask class) have roughly similar mean values across all channels, but the non-cloud pixels stand out in the infrared (IR) band. For clear-sky pixels, the IR band's mean intensity is higher than in any other channel. Next, we visualized several images alongside their masks and examined each spectral channel in detail. During this review—illustrated by the examples below—we discovered some mislabeled samples, reminding us that the task is more challenging than it initially appeared.



Examples of mislabeled data:



This represents only a small margin of the mislabeled data, we estimate the mislabeled data to be above 5%.

#### IV. PREPROCESSING

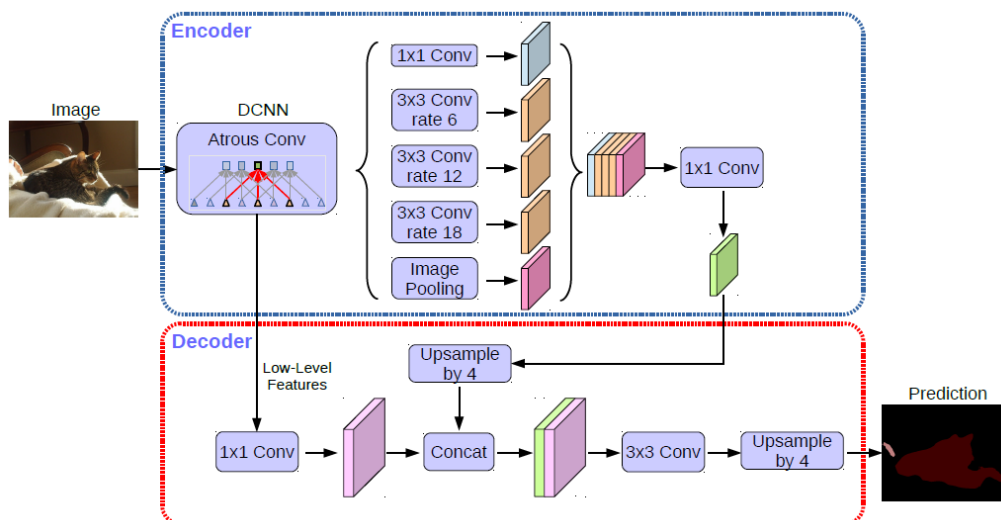
Much of our preprocessing effort went into adapting the dataset to our PyTorch pipeline. On top of the required formatting steps, we experimented with a variety of augmentations—rotations, flips, scaling, and even noise injection. In practice, however, adding noise proved impractical, since it would alter pixel values in the masks and necessitate relabeling, so we ultimately dropped that augmentation.

#### V. MODEL TRAINING

We started our experiments with a plain U-Net—both because it was covered in our lectures and it's a solid baseline—and achieved an average validation Dice score of 0.80. Building on that, we evaluated several more advanced segmentation backbones: U-Net++, DeepLabV3, SegFormer, and DeepLabV3+. Their mean validation Dice scores are summarized below.

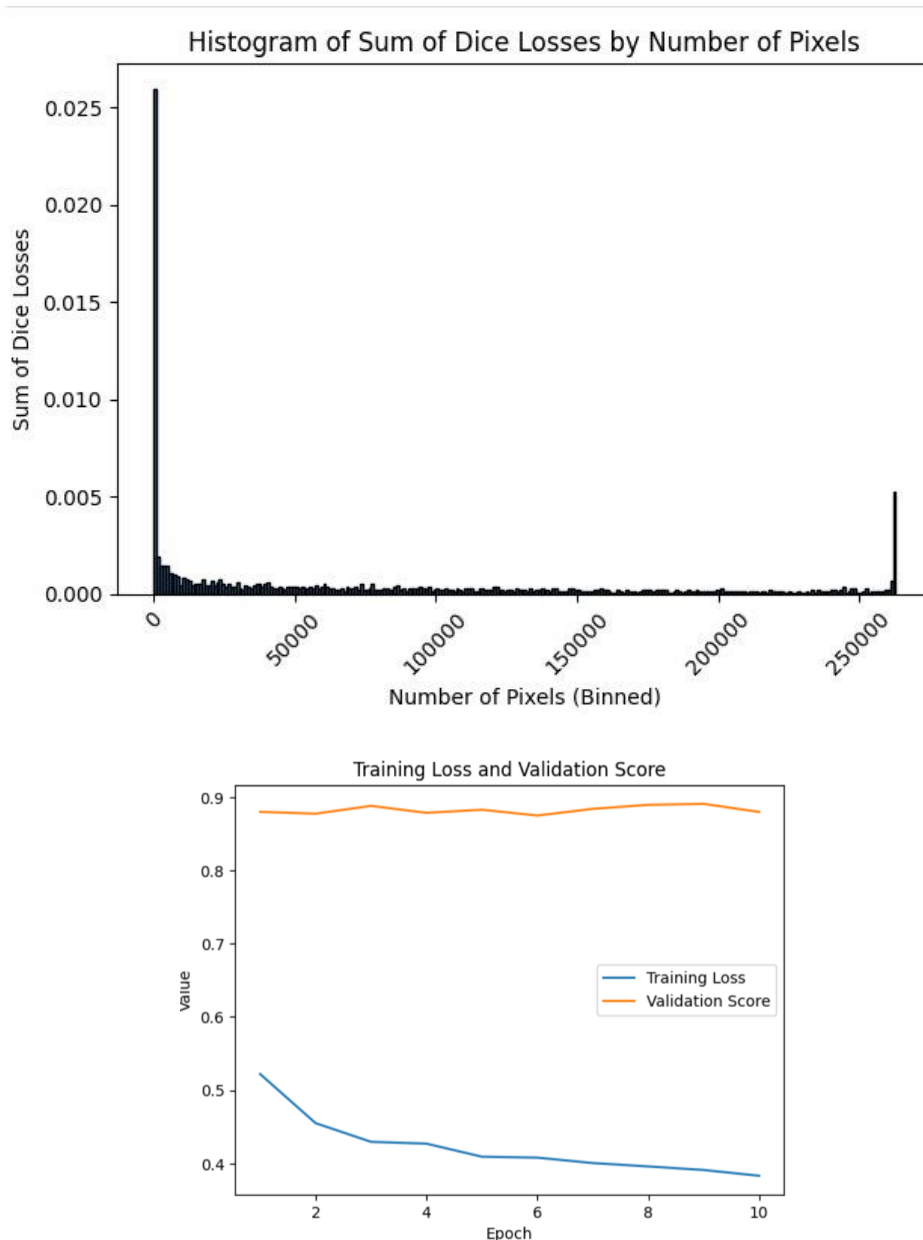
| Model Name | Mean Dice Score |
|------------|-----------------|
| U-Net      | 0.80            |
| U-Net++    | 0.83            |
| DeepLabV3  | 0.86            |
| SegFormer  | 0.87            |
| DeepLabV3+ | 0.89            |

From the experiments we settled on DeepLabV3+, the architecture of the model is provided below



To put it briefly, DeepLabV3+ combines a dilated-convolution encoder with a lightweight decoder to produce sharp, accurate segmentations. First, the backbone extracts features using atrous convolutions to preserve spatial resolution. Those features feed into an Atrous Spatial Pyramid Pooling (ASPP) module, which pools information at multiple dilation rates for rich, multi-scale context. The decoder then upsamples the ASPP output and fuses it with earlier high-resolution features via skip connections, refining object boundaries before the final  $1 \times 1$  convolution produces per-pixel class scores.

We implemented our training loop using a composite loss, binary cross-entropy plus Dice loss, as recommended in the literature, and trained with the Adam optimizer after confirming it outperformed alternatives like SGD and RMSprop. At each epoch we logged both training and validation losses to monitor convergence, and to pinpoint where the model struggled, we plotted the distribution of per-mask Dice scores across the validation set, revealing which images consistently yielded high or low overlap.

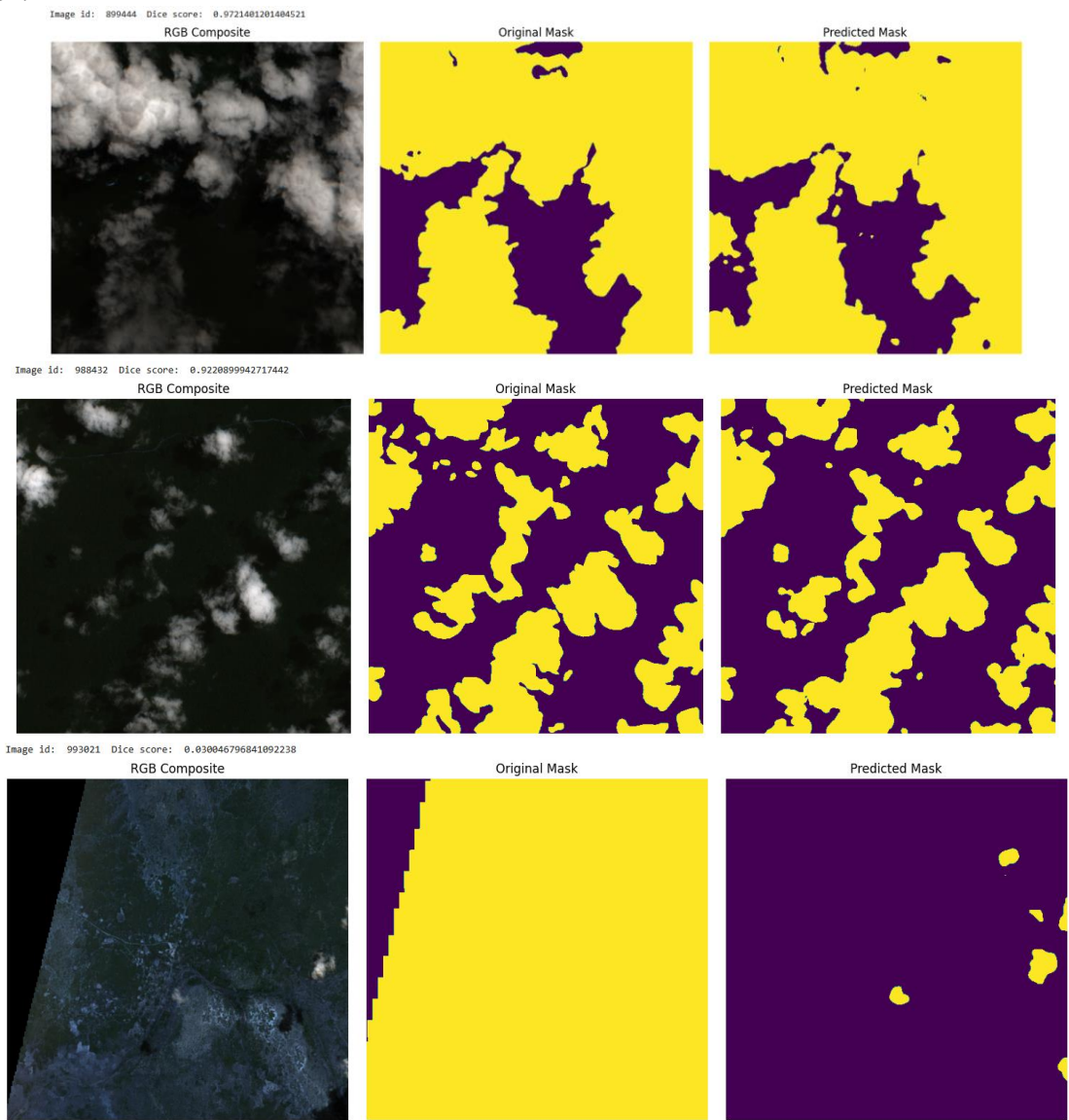


We tuned three key hyperparameters. First, batch size: pushing it above 12 triggered GPU out-of-memory errors, and in practice performance plateaued once we exceeded just four samples per batch. Second, learning rate: values below  $1 \times 10^{-4}$  offered no real gains, while anything substantially higher made convergence unstable and slowed learning. Finally, for converting predicted probabilities into binary masks, we swept thresholds from 0.3 to 0.7 and found that a cutoff around 0.5–0.6 consistently maximized Dice performance.

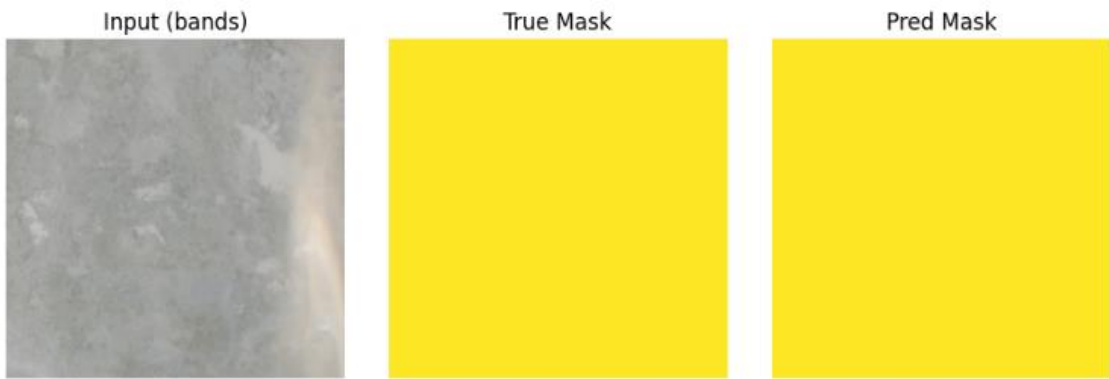
During training, our DeepLabV3+ model plateaued at a Dice score of 0.87, which prompted us to suspect label noise. We therefore inspected the worst-performing validation samples, those with the lowest per-mask Dice scores, and manually corrected any mislabeled pixels, using the model's own predictions as a guide to speed relabeling. After cleaning up these noisy annotations, the model's performance jumped markedly, climbing to a 0.90 Dice score on the validation set.

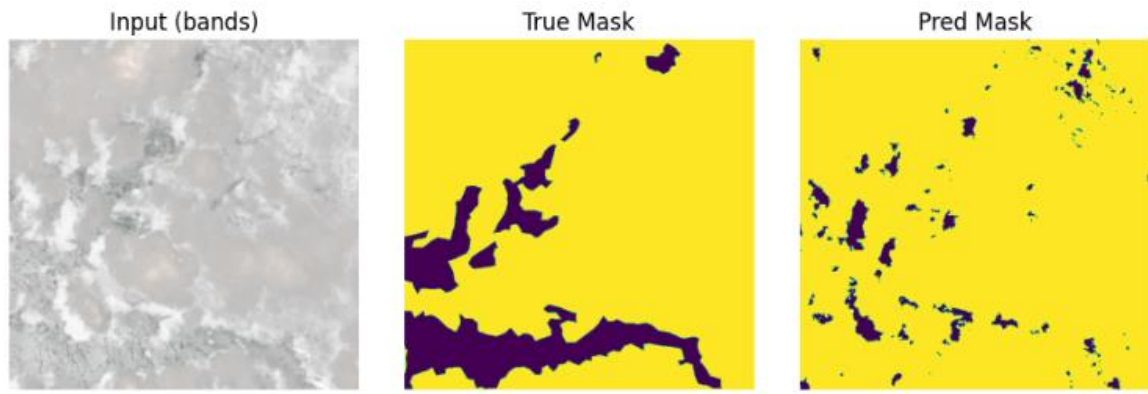
Regarding the classical model a simple logistic regression was used, where the output depended solely on the 4 channels of a certain pixel, surprisingly, the results were somewhat good, achieving a dice score of almost 0.82. Some sample outputs of both models are shown below

DeepLabV3+:



Logistic Regression model:





## VI. PERFORMANCE ANALYSIS

For performance analysis, Dice score was heavily relied on, plots were generated including the ones above.

## VII. REFERENCES

- Chen, L.-C., Zhu, Y., Papandreou, G., Schroff, F., & Adam, H. (2018). *Encoder–Decoder with Atrous Separable Convolution for Semantic Image Segmentation*. In *Proceedings of the European Conference on Computer Vision (ECCV)*. <https://arxiv.org/abs/1802.02611>
- Loddo, F., Piga, D., Michelucci, U., & El Ghazouali, S. (2024). *BenchCloudVision: A Benchmark Analysis of Deep Learning Approaches for Cloud Detection and Segmentation in Remote Sensing Imagery*. arXiv preprint arXiv:2402.13918. <https://arxiv.org/abs/2402.13918>